

DD: SUB-WF-MIGRATION-01-FLOW-WIK — KE WIK

Migration BPMN

Developer implementation guide: DD_SUB-WF-MIGRATION-01-FLOW-WIK-DEV_IMPL_GUIDE-v1.1.md.

Coherence note (2026-06-01): this WIK flow handles same-technology migration directly through WO SHIFTING.

Cross-technology admin migration is delegated from the parent SUB-WF operation to FUL-10. Do not add a second WO migration engine inside this flow.

1. Purpose

This rewritten DD is the developer-facing version of the operator BPMN flow

DD_SUB-WF-MIGRATION-01-FLOW-WIK-v1.1.md.

Verdict: ■ Updated | **Wave:** 1 | **Group I:** Subscription Management | **Version:** v1.1 | **Satellite of:**

DD_SUB-WF-MIGRATION-01-Subscription_Migration-v1.1 KE WIK reference BPMN for subscription migration. Process key: sub-migrate-wik. Composes 12 workers, 3 message catches (cancel + payment + WO-finalized), 1 Camunda intermediate timer (scheduled wait), 1 Drools package. Mirrors RELOCATION FLOW-WIK v1.1 with two additions: Package-change validation/billing context AND a MODIFY_SERVICE step after target ACTIVATE (per-Service-class fan-out of the diff between source and target Package compositions). **What changed from v1.0:** - **Same WO-01 SHIFTING corrections as RELOCATION FLOW-WIK v1.1** — single wo-create-shifting worker, WorkOrderFinalized event catch, WorkOrderCancelled + WorkOrderTimedOut boundary catches, no fabricated disconnection-completed / installation-completed / wo-create-disconnect / wo-create-install. - **Workers reduced from 18 → 12** (equipment binding + WO state-management moved to WO-01's domain; MODIFY_SERVICE step preserved). - **Real framework events.** No disconnect_wo_timeout_days / install_wo_timeout_days config — single wo_kind_config(kind=SHIFTING).default_timeout_days.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/04_subscription_workflows/DD_SUB-WF-MIGRATION-01-FLOW-WIK-v1.1.md
Version	v1.1
DD type	operator BPMN flow
BPMN/process keys	sub-migrate-wik
Drools packages	rules.subscription.migration.wik

4. Runtime Contracts To Implement

4.1 APIs

- POST /api/subscriptions/sub_01HZ_OTIENO_FIBER/migrate
- POST /api/work-orders

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- bil01-emit-intent
- drools-decide
- ful-call-activate
- ful-call-claim-homepass
- ful-call-modify-service
- ful-call-release-homepass
- ful-call-resume
- ful-call-suspend
- fulfillment-completed
- fulfillment-failed
- kafka-emit
- notification-send
- payment-await
- payment-confirmed
- resolve-cycle-anchor-policy
- resolve-disconnect-mode
- resolve-effective-timing
- resolve-equipment-path
- sub-lm-commit-state
- sub-lm-put-master-fields
- sub-lm-put-pending-status
- sub-lm-revert-to-prior
- sub-migrate-wik
- subscription-operation-finalize
- subscription-workflow-rules
- validate-preconditions
- validate-target-homepass
- validate-target-package
- wo-create-shifting

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- SubscriptionMigrated
- SubscriptionMigrationCancelled
- WorkOrderCancelled
- WorkOrderFinalized
- WorkOrderTimedOut

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ Updated | **Wave:** 1 | **Group I:** Subscription Management | **Version:** v1.1 | **Satellite of:** DD_SUB-WF-MIGRATION-01-Subscription_Migration-v1.1

KE WIK reference BPMN for subscription migration. Process key: sub-migrate-wik. Composes 12 workers, 3 message catches (cancel + payment + WO-finalized), 1 Camunda intermediate timer (scheduled wait), 1 Drools package. Mirrors RELOCATION FLOW-WIK v1.1 with two additions: Package-change validation/billing context AND a MODIFY_SERVICE step after target ACTIVATE (per-Service-class fan-out of the diff between source and target Package compositions).

What changed from v1.0:

- **Same WO-01 SHIFTING corrections as RELOCATION FLOW-WIK v1.1** — single wo-create-shifting worker, WorkOrderFinalized event catch, WorkOrderCancelled + WorkOrderTimedOut boundary catches, no fabricated disconnection-completed / installation-completed / wo-create-disconnect / wo-create-install.

- **Workers reduced from 18 → 12** (equipment binding + WO state-management moved to WO-01's domain; MODIFY_SERVICE step preserved).

- **Real framework events.** No disconnect_wo_timeout_days / install_wo_timeout_days config — single wo_kind_config(kind=SHIFTING).default_timeout_days.

1. What this flow is

KE WIK's BPMN composes framework workers to migrate a subscription from source HomePass + source Package to target HomePass + target Package, atomically, while keeping the customer ACTIVE on source until target activation succeeds.

The flow shape mirrors RELOCATION FLOW-WIK v1.1 — request validation, scheduled-wait timer, billing intent with cycle-anchor handling, pay-first gate, source suspend, source HomePass release, physical work (SHIFTING WO or self-move), target activate, **MODIFY_SERVICE diff dispatch**, target HomePass claim, master fields commit, state flip, event emission, notification.

Two flow-specific additions versus RELOCATION:

1. The billing intent is MIGRATION_INTENT with cycleAnchorPolicy in context. BIL-01 v1.1 may return up to 4 line items (MIGRATION_FEE, signed MIGRATION_PRORATE_DELTA for PRESERVE, or OLD_CYCLE_CREDIT + FIRST_CYCLE_CHARGE for RESET, plus optional EQUIPMENT_REPLACEMENT_FEE).
2. After target ACTIVATE_SERVICE succeeds, the BPMN dispatches FUL-03 v1.1's MODIFY_SERVICE with diff between source and target Package compositions. FUL-03 fans out per-service-class add/drop/modify calls to provisioners. This step handles the Package-tier change at the target (e.g., adding IPTV service class, dropping a service class, modifying speed/voice profiles).

Workers composed (12)

#	Worker topic	Used for
1	drools-decide	Validation + resolution: preconditions, target-Package, target-HomePass capability for TARGET Package, equipment-path inference, disconnect-mode, cycle-anchor, effective-timing
2	sub-lm-put-pending-status	Flip to PENDING_MIGRATION (at commit window)
3	bil01-emit-intent	MIGRATION_INTENT — fee + cycle-anchor-aware proration
4	payment-await	Pay-first gate when totals.net > 0
5	ful-call-suspend	FUL-04 SUSPEND_SERVICE at source HomePass with source Package composition
6	ful-call-release-homepass	FUL-07 RELEASE on source HomePass
7	wo-create-shifting	Create SHIFTING WO via WO-01 (conditional — disconnectMode = TECH_DISPATCH only)
8	ful-call-activate	FUL-03 ACTIVATE_SERVICE at target HomePass with TARGET Package composition
9	ful-call-modify-service	FUL-03 MODIFY_SERVICE at target — diff source vs target Package compositions, fan out per-service-class adjustments
10	ful-call-claim-homepass	FUL-07 CLAIM on target HomePass

#	Worker topic	Used for
11	sub-lm-put-master-fields	Write homepass_id, previous_homepass_id, package_ref, package_version_id, previous_package_ref, previous_package_version_id, last_migrated_at, last_status_changed_at (single atomic call)
12	sub-lm-commit-state	Flip subscription back to ACTIVE

Plus framework-standard kafka-emit, notification-send, subscription-operation-finalize workers.

Message catches + timers

Type	Name / time expression	Used when
Intermediate timer	timeDate=\${scheduledCommitAt}	Non-IMMEDIATE wait
Boundary message on timer	external_cancel_requested (correlated on operationId)	Cancellation during scheduled wait
Intermediate message catch	payment-confirmed (correlated on operationId)	Pay-first gate
Boundary timer on payment catch	timeDuration=P7D	Payment timeout → cancel migration
Intermediate message catch	WorkOrderFinalized (correlated on callerContext.operationId)	After wo-create-shifting; skipped when disconnectMode = CUSTOMER_SELF_MOVE
Boundary message on WO catch	WorkOrderCancelled (correlated on callerContext.operationId)	Dispatcher force-cancels WO
Boundary message on WO catch	WorkOrderTimedOut (correlated on callerContext.operationId)	WO exceeds wo_kind_config(kind=SHIFTING).default_timeout_days
Intermediate message catch	fulfillment-completed / fulfillment-failed (correlated on fulfillmentCorrelationId)	After each FUL-03/FUL-04/FUL-07 call

Drools package

rules.subscription.migration.wik — declared decisions:

Decision	Output	Used at
validate-preconditions	eligible: bool, eligibilityReasonCode?, sourceSnapshots: object	Subscription ACTIVE, no in-flight ops, role-trigger match, gated trigger check
validate-target-package	targetPackageValid: bool, targetPackageReasonCode?, targetPackageSnapshot: object	Target Package differs from source, sellable, same billing-mode + currency
validate-target-homepass	targetHomepassValid: bool, targetHomepassReasonCode?, targetHomepassSnapshot: object, missingServiceManagementKeys: array, unprovisionedPorts: array, sameFranchise: bool, networkRootTypeChanged: bool	Target HomePass sellable + supports TARGET Package's required service families + ports resolved; cross-tech check for CUSTOMER_REQUESTED_MIGRATION
resolve-equipment-path	resolvedEquipmentPath: str, equipmentDelta: object, pathReasonCode?	Auto-infer from network-tech compatibility + equipment requirements delta
resolve-disconnect-mode	resolvedDisconnectMode: str, disconnectModeReasonCode?	NEW_INSTALL forces TECH_DISPATCH; else request or config default
resolve-cycle-anchor-policy	resolvedCycleAnchorPolicy: str, cycleAnchorReasonCode?	Request or operator-config default; incompatibility check with effective-timing
resolve-effective-timing	resolvedEffectiveTiming: str, scheduledCommitAt: timestamp?, timingValid: bool, timingReasonCode?	Request or default; bound validation

2. BPMN structure

Process key: sub-migrate-wik. Deployed as Subscription_Migration_WIK.bpmn.

2.1 Outline

```
START (event)
■ Process variables from trigger endpoint:
■ operationId, subscriptionId, customerId, operatorCode, sourceHomepassId,
■ sourcePackageRef, sourcePackageVersionId,
■ migrationTrigger, targetHomepassId, targetPackageRef,
■ equipmentPath?, disconnectMode?, cycleAnchorPolicy?,
■ effectiveTiming?, scheduledAt?, notes,
■ initiatingActorUserId, initiatingActorRole, initiatingWorkflowKind,
■ initiatingWorkflowRef, idempotencyKey
■
▼
[STATE: VALIDATING_REQUEST]
Service Task: drools-decide
  inputs: kjarPackage="rules.subscription.migration.wik",
         factsJson={subscription state, source Package snapshot, source HomePass snapshot,
                    target Package snapshot, target HomePass snapshot,
                    equipmentPath request, disconnectMode request,
                    cycleAnchorPolicy request,
                    effectiveTiming + scheduledAt request,
                    role, operator config},
         requestedDecisions=["validate-preconditions", "validate-target-package",
                             "validate-target-homepass", "resolve-equipment-path",
                             "resolve-disconnect-mode", "resolve-cycle-anchor-policy",
                             "resolve-effective-timing"]
  outputs: <all 7 decisions' outputs>
  ■
  ▼ XOR: all checks pass?
  ■ NO → REJECT subflow (emit SubscriptionMigrationRejected + finalize FAILED)
  ■ YES ↓
  ■
  ▼ XOR: resolvedEffectiveTiming == "IMMEDIATE"?
  ■ YES ↓
  ■ NO ↓
  ■ [STATE: SCHEDULED]
  ■ Service Task: kafka-emit
  ■   inputs: topic="sophix.subscription.subscription.migrationscheduled",
  ■          payloadJson=<SubscriptionMigrationScheduled per parent $6>
  ■   ▼
  ■ Service Task: notification-send (reliability-isolated)
  ■   inputs: notificationEventKind="SUBSCRIPTION_MIGRATION_SCHEDULED",
  ■          eventContext={sourceHomepassId, targetHomepassId, targetPackageRef,
  ■                      scheduledCommitAt, resolvedEquipmentPath, sameFranchise}
  ■   ▼
  ■ Intermediate Catch Event: timer
  ■     timeDate: ${scheduledCommitAt}
  ■     Boundary listener: external_cancel_requested
  ■     → on cancel: emit SubscriptionMigrationCancelled + finalize CANCELLED
  ■   ▼ timer fires
  ■
  ▼
[STATE: VALIDATING (commit-time revalidation)]
Service Task: drools-decide (revalidation)
  inputs: <re-loaded current snapshots>,
         requestedDecisions=["validate-preconditions", "validate-target-package",
                             "validate-target-homepass", "resolve-equipment-path",
                             "resolve-disconnect-mode", "resolve-cycle-anchor-policy"]
  ■
  ▼ XOR: revalidation passes?
  ■ NO → REJECT (rejectionStage="COMMIT_REVALIDATION")
  ■ YES ↓
  ■
[STATE: PENDING_STATE_FLIP]
Service Task: sub-lm-put-pending-status
  inputs: subscriptionId, toStatus="PENDING_MIGRATION",
         currentTransitionType="MIGRATION",
         currentTransitionReasonCode=<migrationTrigger>
  ■
  ▼
[STATE: BILLING_CALL]
Service Task: bil01-emit-intent
  inputs: subscriptionId,
         intentCode="MIGRATION_INTENT",
         intentContext={sourceHomepassId, sourceFranchiseId, sourceNetworkRootType,
                        sourcePackageRef, sourcePackageVersionId, sourcePackagePrice,
                        targetHomepassId, targetFranchiseId, targetNetworkRootType,
                        sameFranchise, targetPackageRef, targetPackageVersionId,
                        targetPackagePrice, currency,
                        equipmentPath=<resolvedEquipmentPath>,
                        disconnectMode=<resolvedDisconnectMode>,
                        migrationTrigger,
```

```

        cycleAnchorPolicy=<resolvedCycleAnchorPolicy>,
        effectiveTiming=<resolvedEffectiveTiming>,
        scheduledCommitAt?,
        cycleWindow}
outputs: intentDecisionId, intentDecision (lineItems[] + totals)
■
▼ XOR: totals.net > 0 AND pay_first_required?
■ NO ↓
■ YES ↓
■ [STATE: AWAITING_PAYMENT]
■ Intermediate Catch Event: payment-confirmed (correlated on operationId)
■ Boundary timer: P7D
■ → on timeout: emit SubscriptionMigrationRejected (PAYMENT_TIMEOUT)
■ + revert state to ACTIVE + finalize FAILED
■
▼
[STATE: SUSPENDING_SOURCE]
Service Task: ful-call-suspend
inputs: subscriptionId, homepassId=<sourceHomepassId>, customerId,
        intent="SUSPEND_SERVICE",
        reasonCategory="SOURCE_DECOMMISSION",
        currentPackageCompositionSnapshot=<sourcePackageComposition>,
        sourceServiceManagementEndpoints,
        fulfillmentCorrelationId=<operationId>:suspend-source
outputs: suspendFulfillmentId, suspendTerminalState
■
▼ Message catch: fulfillment-completed | fulfillment-failed
■ FAILED → REVERT subflow
■ COMPLETED ↓
■
[STATE: RELEASING_SOURCE_HOMEPASS]
Service Task: ful-call-release-homepass
inputs: subscriptionId, homepassId=<sourceHomepassId>, customerId,
        releaseReason="MIGRATION",
        releasePortBinding=<if AT_INSTALLATION then true else false>,
        fulfillmentCorrelationId=<operationId>:release-source
outputs: releaseFulfillmentId
■
▼ Message catch: fulfillment-completed | fulfillment-failed
■ FAILED → REVERT subflow
■ COMPLETED ↓
■
▼ XOR: resolvedDisconnectMode == "TECH_DISPATCH"?
■ NO (CUSTOMER_SELF_MOVE) ↓
■ Log self-move acknowledgement step (audit only)
■ Variable finalEquipmentBindings := <currentEquipmentBindings>
■ YES ↓
■ [STATE: AWAITING_WO_FINALIZED]
■ Service Task: wo-create-shifting
■ inputs: subscriptionId, customerId, operatorCode,
■ kind="SHIFTING",
■ jobTypeCode=<resolved by WO-01 Drools from sourceNetworkRootType + equipmentPath>,
■ homepassRefs={primary: <targetHomepassId>, secondary: <sourceHomepassId>},
■ currentEquipmentBindings,
■ scheduledAt=<NOW for IMMEDIATE; or original scheduledAt for SCHEDULED_AT>,
■ flowContext={equipmentPath: <resolvedEquipmentPath>,
■ disconnectMode: <resolvedDisconnectMode>,
■ sourceHomepassId, targetHomepassId,
■ equipmentDelta},
■ callerContext={subscriptionId, operationId,
■ callerWorkflowKind: "SUB_WF_MIGRATION",
■ callerWorkflowRefId: operationId}
■ outputs: shiftingWoId
■
▼
■ Intermediate Catch Event: WorkOrderFinalized
■ Correlation: callerContext.operationId
■ Payload includes: finalEquipmentBindings, accountSubStatusChange, woFinalizedAt
■ Boundary message: WorkOrderCancelled (correlated on callerContext.operationId)
■ → emit SubscriptionMigrationRejected (WO_CANCELLED_BY_DISPATCHER)
■ + REVERT + finalize FAILED
■ Boundary message: WorkOrderTimedOut (correlated on callerContext.operationId)
■ → emit SubscriptionMigrationRejected (WO_TIMEOUT)
■ + REVERT + finalize FAILED
■
▼
[STATE: ACTIVATING_TARGET]
Service Task: ful-call-activate
inputs: subscriptionId, homepassId=<targetHomepassId>, customerId,
        intent="ACTIVATE_SERVICE",
        packageRef=<targetPackageRef>,
        packageVersionId=<targetPackageVersionId>,
        packageCompositionSnapshot=<targetPackageComposition>,

```

```

        targetServiceManagementEndpoints,
        equipmentBindings=<finalEquipmentBindings>,
        fulfillmentCorrelationId=<operationId>:activate-target
    outputs: activateFulfillmentId, activateTerminalState
    ■
    ▼ Message catch: fulfillment-completed | fulfillment-failed
    ■ FAILED → REVERT subflow
    ■ COMPLETED ↓
    ■
[STATE: MODIFYING_SERVICE]
Service Task: ful-call-modify-service
    inputs: subscriptionId, homepassId=<targetHomepassId>, customerId,
           intent="MODIFY_SERVICE",
           sourcePackageCompositionSnapshot,
           targetPackageCompositionSnapshot,
           targetServiceManagementEndpoints,
           equipmentBindings=<finalEquipmentBindings>,
           fulfillmentCorrelationId=<operationId>:modify-target
    outputs: modifyFulfillmentId, modifyTerminalState
    ■ FUL-03 internally diffs source vs target compositions, dispatches add/drop/modify
    ■ per service class via the per-service-class fan-out (PLM-CFG-01 service_management_endpoints)
    ■
    ▼ Message catch: fulfillment-completed | fulfillment-failed
    ■ FAILED → manual recovery (service activated at target but Package-tier delta not applied
    ■ – operational alert)
    ■ COMPLETED ↓
    ■
[STATE: CLAIMING_TARGET_HOMEPASS]
Service Task: ful-call-claim-homepass
    inputs: subscriptionId, homepassId=<targetHomepassId>, customerId,
           claimReason="MIGRATION",
           portBindings=<from finalEquipmentBindings if AT_INSTALLATION>,
           fulfillmentCorrelationId=<operationId>:claim-target
    outputs: claimFulfillmentId
    ■
    ▼ Message catch: fulfillment-completed | fulfillment-failed
    ■ FAILED → manual recovery
    ■ COMPLETED ↓
    ■
[STATE: COMMITTING_FINAL_STATE]
Service Task: sub-lm-put-master-fields
    inputs: subscriptionId,
           fieldsJson={
               homepass_id: <targetHomepassId>,
               previous_homepass_id: <sourceHomepassId>,
               package_ref: <targetPackageRef>,
               package_version_id: <targetPackageVersionId>,
               previous_package_ref: <sourcePackageRef>,
               previous_package_version_id: <sourcePackageVersionId>,
               last_migrated_at: NOW,
               last_status_changed_at: NOW
           }
    ■
    ▼
Service Task: sub-lm-commit-state
    inputs: subscriptionId, toStatus="ACTIVE", expectedFromStatus="PENDING_MIGRATION"
    ■
    ▼
[STATE: EMITTING_EVENT]
Service Task: kafka-emit
    inputs: topic="sophix.subscription.subscription.migrated",
           key=<subscriptionId>,
           payloadJson=<full SubscriptionMigrated payload per parent $6>
    ■
    ▼
Service Task: notification-send (reliability-isolated)
    inputs: notificationEventKind="SUBSCRIPTION_MIGRATED",
           eventContext={sourceHomepassId, targetHomepassId,
                        sourcePackageRef, targetPackageRef, sameFranchise,
                        resolvedEquipmentPath, resolvedDisconnectMode,
                        shiftingWoId, finalEquipmentBindings,
                        actualCommitAt, migrationTrigger,
                        billingDetails}
    ■
    ▼
[STATE: COMPLETED]
Service Task: subscription-operation-finalize
    inputs: operationId, finalState="COMPLETED"
    ■
END (event)

```


2.2 BPMN XML excerpts

SHIFTING WO message catch with cancel + timeout boundaries (same pattern as RELOCATION FLOW-WIK):

```
<bpmn:intermediateCatchEvent id="Catch_wo_finalized">
  <bpmn:messageEventDefinition messageRef="Message_WorkOrderFinalized" />
</bpmn:intermediateCatchEvent>

<bpmn:boundaryEvent id="Boundary_wo_cancelled" attachedToRef="Catch_wo_finalized" cancelActivity="true">
  <bpmn:messageEventDefinition messageRef="Message_WorkOrderCancelled" />
</bpmn:boundaryEvent>

<bpmn:boundaryEvent id="Boundary_wo_timedout" attachedToRef="Catch_wo_finalized" cancelActivity="true">
  <bpmn:messageEventDefinition messageRef="Message_WorkOrderTimedOut" />
</bpmn:boundaryEvent>
```

Message correlation: `${callerContext.operationId}` extracted from event payload matches BPMN process variable `operationId`.

MODIFY_SERVICE step:

```
<bpmn:serviceTask id="Task_modify_service" name="FUL-03 MODIFY_SERVICE diff dispatch"
  camunda:type="external" camunda:topic="ful-call-modify-service">
  <bpmn:extensionElements>
    <camunda:inputOutput>
      <camunda:inputParameter name="intent">MODIFY_SERVICE</camunda:inputParameter>
      <camunda:inputParameter name="sourcePackageComposition">${sourcePackageComposition}</camunda:inputParameter>
      <camunda:inputParameter name="targetPackageComposition">${targetPackageComposition}</camunda:inputParameter>
    </camunda:inputOutput>
  </bpmn:extensionElements>
</bpmn:serviceTask>
```

2.3 Compensation paths

If this step fails	Compensation runs
drools-decide (request OR commit revalidation)	No mutations; emit Rejected; finalize FAILED
Scheduled wait — external cancel	Emit Cancelled; finalize CANCELLED
sub-lm-put-pending-status	Atomic; failure leaves subscription on source; emit Rejected; finalize FAILED
bil01-emit-intent	sub-lm-revert-to-prior; emit Rejected; finalize FAILED
Pay-first timeout (7 days)	sub-lm-revert-to-prior; emit Rejected (PAYMENT_TIMEOUT); finalize FAILED
ful-call-suspend at source	sub-lm-revert-to-prior; emit Rejected; finalize FAILED
ful-call-release-homepass at source	ful-call-resume at source (best-effort); sub-lm-revert-to-prior; emit Rejected; finalize FAILED
wo-create-shifting fails	FUL-07 CLAIM back on source (best-effort); FUL-04 RESUME at source (best-effort); sub-lm-revert-to-prior; emit Rejected; finalize FAILED
WorkOrderCancelled	FUL-07 CLAIM back on source; FUL-04 RESUME at source — best-effort; sub-lm-revert-to-prior; emit Rejected (WO_CANCELLED_BY_DISPATCHER); finalize FAILED
WorkOrderTimedOut	Same as WorkOrderCancelled; emit Rejected (WO_TIMEOUT)
ful-call-activate at target	FUL-07 CLAIM back on source (best-effort); FUL-04 RESUME at source (best-effort); revert state; emit Rejected. WO already finalized — equipment installed at target — operational alert.
ful-call-modify-service at target	Manual recovery — target ACTIVE on source Package composition; tier-delta not applied; operational alert. Subscription state reflects: homepass_id still source, package_ref still source (master fields not yet committed).
ful-call-claim-homepass at target	Manual recovery — target service running but FUL-07 binding not opened.
sub-lm-put-master-fields	Manual recovery — service running but master fields not updated.
sub-lm-commit-state	Manual recovery.
kafka-emit	Outbox dispatcher retries.
notification-send	Logged but not compensated.

2.4 Configuration row (KE WIK)

In `subscription_operation_config` (framework parent):

operator_code	operation_kind	default_bpmn_process_key	operation_timeout_seconds	feature_flags
WIK	MIGRATION	sub-migrate-wik	5184000	{}

In subscription_migration_config (parent §3.1):

operator_code	customer_self_service_enabled	pay_first_required	default_effective_timing	max_future_scheduled_days	default_disconnect_mode_key_existing	default_cycle_anchor_policy	admin_tech_migration_enabled	customer_notification_enabled
WIK	true	true	SCHEDULED_A T	60	TECH_DISCONNECT	PRESERVE	true when FUL-10 is enabled for an approved migration cohort; otherwise false	true

In WO-01's wo_kind_config (referenced):

operator_code	kind	default_timeout_days
WIK	SHIFTING	14

3. Drools package rules.subscription.migration.wik

KE WIK rules deployed as part of subscription-workflow-rules KJAR. Same patterns as RELOCATION FLOW-WIK Drools with migration-specific additions.

Key rules (signatures only; full bodies follow same pattern as RELOCATION FLOW-WIK §3):

3.1 validate-preconditions

Same shape as RELOCATION FLOW-WIK 3.1, plus:

```
rule "ADMIN_TECH_MIGRATION requires FUL-10 operator enablement"
when
    $req: MigrationRequestFact(migrationTrigger == "ADMIN_TECH_MIGRATION", eligible == null)
    $cfg: OperatorConfigFact(adminTechMigrationEnabled == false)
then
    modify($req) {
        setEligible(false);
        setEligibilityReasonCode("TECHNOLOGY_MIGRATION_NOT_ENABLED")
    }
end
```

3.2 validate-target-package

```
rule "Target Package: same as source"
when
    $req: MigrationRequestFact(eligible == true, targetPackageValid == null,
                               targetPackageRef == sourcePackageRef)
then
    modify($req) {
        setTargetPackageValid(false);
        setTargetPackageReasonCode("SAME_PACKAGE")
    }
end

rule "Target Package: not sellable"
when
    $req: MigrationRequestFact(eligible == true, targetPackageValid == null)
    $tpkg: PackageFact(packageRef == $req.targetPackageRef, isSellable == false)
then
    modify($req) {
        setTargetPackageValid(false);
        setTargetPackageReasonCode("TARGET_PACKAGE_NOT_SELLABLE")
    }
end

rule "Target Package: billing-mode mismatch"
when
    $req: MigrationRequestFact(eligible == true, targetPackageValid == null)
    $tpkg: PackageFact(packageRef == $req.targetPackageRef)
    $spkg: PackageFact(packageRef == $req.sourcePackageRef)
    eval(!$tpkg.getBillingMode().equals($spkg.getBillingMode()))
then
```

```

    modify($req) {
        setTargetPackageValid(false);
        setTargetPackageReasonCode("BILLING_MODE_MISMATCH")
    }
end

rule "Target Package: currency mismatch"
when
    $req: MigrationRequestFact(eligible == true, targetPackageValid == null)
    $tpkg: PackageFact(packageRef == $req.targetPackageRef)
    $spkg: PackageFact(packageRef == $req.sourcePackageRef)
    eval(!$tpkg.getCurrency().equals($spkg.getCurrency()))
then
    modify($req) {
        setTargetPackageValid(false);
        setTargetPackageReasonCode("CURRENCY_MISMATCH")
    }
end

rule "Target Package: default-pass"
salience -100
when
    $req: MigrationRequestFact(eligible == true, targetPackageValid == null)
    $tpkg: PackageFact(packageRef == $req.targetPackageRef)
then
    modify($req) {
        setTargetPackageValid(true);
        setTargetPackageSnapshot($tpkg)
    }
end

```

3.3 validate-target-homepass

Same pattern as RELOCATION FLOW-WIK 3.2, but checks TARGET Package's required service-management keys (not source's), plus a cross-tech check for CUSTOMER_REQUESTED_MIGRATION:

```

rule "Target HomePass: cross-tech with customer-requested trigger is rejected"
when
    $req: MigrationRequestFact(eligible == true, targetPackageValid == true,
        targetHomepassValid == null,
        migrationTrigger == "CUSTOMER_REQUESTED_MIGRATION")
    eval(!$req.getSourceNetworkRootType().equals($req.getTargetNetworkRootType()))
then
    modify($req) {
        setTargetHomepassValid(false);
        setTargetHomepassReasonCode("CROSS_TECH_CUSTOMER_REQUEST_NOT_ALLOWED");
        setNetworkRootTypeChanged(true)
    }
end

```

3.4 resolve-equipment-path

Same as RELOCATION FLOW-WIK 3.3, plus equipment-delta computation:

```

rule "Equipment path: target Package needs new equipment kinds → NEW_INSTALL"
when
    $req: MigrationRequestFact(eligible == true, targetPackageValid == true,
        targetHomepassValid == true,
        resolvedEquipmentPath == null)
    $tpkg: PackageFact(packageRef == $req.targetPackageRef)
    $spkg: PackageFact(packageRef == $req.sourcePackageRef)
    eval(!$spkg.getRequiredEquipmentKinds().containsAll($tpkg.getRequiredEquipmentKinds()))
then
    java.util.Set added = new java.util.HashSet($tpkg.getRequiredEquipmentKinds());
    added.removeAll($spkg.getRequiredEquipmentKinds());
    modify($req) {
        setResolvedEquipmentPath("NEW_INSTALL");
        setEquipmentDelta(new EquipmentDelta(new java.util.ArrayList(added), new java.util.ArrayList()))
    }
end

rule "Equipment path: target Package drops equipment kinds → equipmentDelta tracks obsolete"
when
    $req: MigrationRequestFact(resolvedEquipmentPath != null, equipmentDelta == null)
    $tpkg: PackageFact(packageRef == $req.targetPackageRef)
    $spkg: PackageFact(packageRef == $req.sourcePackageRef)
    eval(!$tpkg.getRequiredEquipmentKinds().containsAll($spkg.getRequiredEquipmentKinds()))
then
    java.util.Set obsolete = new java.util.HashSet($spkg.getRequiredEquipmentKinds());
    obsolete.removeAll($tpkg.getRequiredEquipmentKinds());
    java.util.Set added = new java.util.HashSet($tpkg.getRequiredEquipmentKinds());

```

```

        added.removeAll($spkg.getRequiredEquipmentKinds());
        modify($req) {
            setEquipmentDelta(new EquipmentDelta(new java.util.ArrayList(added), new java.util.ArrayList(obsolete)))
        }
    end

```

3.5 resolve-disconnect-mode

Same as RELOCATION FLOW-WIK 3.4 verbatim.

3.6 resolve-cycle-anchor-policy

```

rule "Cycle anchor: RESET + END_OF_CURRENT_CYCLE is incompatible"
when
    $req: MigrationRequestFact(resolvedCycleAnchorPolicy == null,
                               cycleAnchorPolicy == "RESET_TO_OPERATION_DATE",
                               effectiveTiming == "END_OF_CURRENT_CYCLE")
then
    modify($req) {
        setCycleAnchorReasonCode("INCOMPATIBLE_CYCLE_ANCHOR_AND_EFFECTIVE_TIMING");
        setTargetHomepassValid(false)
    }
end

rule "Cycle anchor: use request value"
when
    $req: MigrationRequestFact(resolvedCycleAnchorPolicy == null,
                               cycleAnchorPolicy != null, cycleAnchorReasonCode == null)
    eval("PRESERVE".equals($req.getCycleAnchorPolicy()) ||
         "RESET_TO_OPERATION_DATE".equals($req.getCycleAnchorPolicy()))
then
    modify($req) { setResolvedCycleAnchorPolicy($req.getCycleAnchorPolicy()) }
end

rule "Cycle anchor: default from operator config"
when
    $req: MigrationRequestFact(resolvedCycleAnchorPolicy == null,
                               cycleAnchorPolicy == null, cycleAnchorReasonCode == null)
    $cfg: OperatorConfigFact()
then
    modify($req) { setResolvedCycleAnchorPolicy($cfg.getDefaultCycleAnchorPolicy()) }
end

```

3.7 resolve-effective-timing

Same as RELOCATION FLOW-WIK 3.5.

4. Worked example — Otieno's same-tech migration with Package upgrade

Context: Otieno is on pkg_INET_100M_VOIP_RES (double-play) at HomePass hp_01HZ_KAREN_017. He's moving to Lavington on June 1 + wants triple-play (pkg_INET_VOIP_IPTV_RES). Same network tech across both HomePasses. PRESERVE cycle-anchor. Target Package adds IPTV — requires STB not on source — forces equipmentPath = NEW_INSTALL.

May 15, 14:00:00 — Trigger:

```

POST /api/subscriptions/sub_01HZ_OTIENO_FIBER/migrate
Idempotency-Key: migrate-sub_01HZ_OTIENO-hp_LAVI_073-pkg_INET_VOIP_IPTV-20260601

{ "operatorCode": "WIK", "migrationTrigger": "CUSTOMER_REQUESTED_MIGRATION",
  "targetHomepassId": "hp_01HZ_LAVI_073",
  "targetPackageRef": "pkg_01HX_INET_VOIP_IPTV_RES",
  "cycleAnchorPolicy": "PRESERVE",
  "effectiveTiming": "SCHEDULED_AT",
  "scheduledAt": "2026-06-01T09:00:00+03:00",
  "initiatingActorUserId": "cc-agent-wik-007",
  "initiatingActorRole": "CALL_CENTER" }

→ 202 Accepted
{ "operationId": "subop_01HZ_OTIENO_MIGRATE",
  "resolvedEquipmentPath": "NEW_INSTALL",
  "resolvedDisconnectMode": "TECH_DISPATCH",
  "resolvedCycleAnchorPolicy": "PRESERVE",
  "resolvedEffectiveTiming": "SCHEDULED_AT",
  "equipmentDelta": { "addedEquipmentKinds": ["STB_IPTV"], "obsoleteEquipmentKinds": [] },

```

```
"scheduledCommitAt": "2026-06-01T09:00:00+03:00",  
"currentState": "SCHEDULED" }
```

May 15, 14:00:00.5 — VALIDATING_REQUEST: Drools runs all 7 decisions. Subscription ACTIVE ✓, no in-flight ✓, role-trigger match ✓, gated trigger check passes (not ADMIN_TECH_MIGRATION). Target Package differs from source ✓, sellable ✓, same billing-mode + currency ✓. Target HomePass status RFS ✓, supports TARGET Package's required keys (data + voice + iptv_multicast all present) ✓, ports non-null ✓. Same network root type → not cross-tech. Target Package adds STB_IPTV → equipmentPath = NEW_INSTALL, equipmentDelta = {addedEquipmentKinds: [STB_IPTV], obsoleteEquipmentKinds: []}. disconnectMode forced TECH_DISPATCH. cycleAnchorPolicy = PRESERVE from request. Effective timing SCHEDULED_AT, within 60 days.

May 15, 14:00:01 — SCHEDULED: Kafka event emitted. Otieno notified.

June 1, 09:00:00 — Timer fires. Revalidation passes.

sub-lm-put-pending-status → PENDING_MIGRATION.

June 1, 09:00:02 — BILLING_CALL: bil01-emit-intent calls BIL-01 v1.1 with MIGRATION_INTENT. BIL-CFG-01 has MIGRATION_FEE matching, MIGRATION_PRORATE_DELTA matching (SIGNED, filter rule matches PRESERVE), EQUIPMENT_REPLACEMENT_FEE matching (filter on equipmentPath = NEW_INSTALL AND replacementReason != null).

For Otieno's case: sourcePackagePrice = 2500.00, targetPackagePrice = 7500.00, daysRemaining = 30, cycleLengthDays = 30. Signed proration: $(7500 - 2500) \times (30/30) = +5000$ (POSITIVE — Otieno owes the difference). Migration fee 1500. Equipment replacement fee: target Package adds STB but no source equipment is being replaced — Drools sets replacementReason = null → this BillableEvent skipped.

```
{  
  "decisionId": "bil_dec_01HZ_OTIENO_MIGRATE",  
  "lineItems": [  
    {"billableEventCode": "MIGRATION_FEE", "amount": 1500.00, "currency": "KES"},  
    {"billableEventCode": "MIGRATION_PRORATE_DELTA", "amount": 5000.00, "currency": "KES",  
     "metadata": {"computedSign": "POSITIVE", "daysRemaining": 30}}  
  ],  
  "totals": {"gross": 6500.00, "tax": 1040.00, "net": 7540.00, "currency": "KES"},  
  "payFirstRequired": true, "settlementStatus": "PENDING"  
}
```

June 1, 09:00:03 — AWAITING_PAYMENT: Pay-first gate. CC agent processes M-Pesa payment. payment-confirmed at 09:00:55.

June 1, 09:00:57 — SUSPENDING_SOURCE: ful-call-suspend → FUL-04 SUSPEND_SERVICE at source HomePass with source Package composition (INET + VOIP). Source service deactivated at 09:01:00.

June 1, 09:01:02 — RELEASING_SOURCE_HOMEPASS: ful-call-release-homepass → FUL-07 RELEASE on source. Source ATP decrement, port bindings released, source homepass_subscription_binding closed. Returns at 09:01:03.

June 1, 09:01:04 — AWAITING_WO_FINALIZED: wo-create-shifting calls WO-01 POST /api/work-orders with kind=SHIFTING, flowContext={equipmentPath:NEW_INSTALL, disconnectMode:TECH_DISPATCH, equipmentDelta:{addedEquipmentKinds:[STB_IPTV], obsoleteEquipmentKinds:[]}, sourceHomepassId, targetHomepassId}, callerContext={operationId, callerWorkflowKind:SUB_WF_MIGRATION}. WO wo_01HZ_OTIENO_SHIFT_5501 created. Tech dispatched. WO-01 SHIFTING flow internally: source ONT + ATA recovered, transported, plus fresh STB picked up from depot; at target ONT + ATA + new STB installed and topology confirmed. WO finalized at 10:55:00 with finalEquipmentBindings: [ONT eqp_ONT_42→INET, ATA eqp_ATA_19→VOIP, STB eqp_STB_new_77→IPTV]. WorkOrderFinalized event arrives at 10:55:02.

June 1, 10:55:04 — ACTIVATING_TARGET: ful-call-activate → FUL-03 ACTIVATE_SERVICE at target HomePass with TARGET Package composition (INET + VOIP + IPTV). FUL-03 dispatches per-service-class provisioning at target OLT (INET), target VOIP switch, target IPTV multicast. Returns at 10:55:08.

June 1, 10:55:10 — MODIFYING_SERVICE: ful-call-modify-service → FUL-03 MODIFY_SERVICE with source composition [INET, VOIP] + target composition [INET, VOIP, IPTV]. FUL-03 internally computes diff: addedServiceClasses=[IPTV], droppedServiceClasses=[], modifiedServiceClasses=[]. Dispatches add-IPTV per-customer setup (IGMP / multicast provisioning, IPTV middleware customer registration). Returns at 10:55:32.

June 1, 10:55:34 — CLAIMING_TARGET_HOMEPASS: ful-call-claim-homepass → FUL-07 CLAIM on target. Target ATP increment, port bindings captured (3 ports — INET data, voice, IPTV multicast). Returns at 10:55:36.

June 1, 10:55:38 — COMMITTING_FINAL_STATE: sub-lm-put-master-fields writes atomically: homepass_id=hp_LAVI_073, previous_homepass_id=hp_KAREN_017,

package_ref=pkg_INET_VOIP_IPTV_RES, package_version_id=pkv_INET_VOIP_IPTV_V1, previous_package_ref=pkg_INET_100M_VOIP_RES, previous_package_version_id=pkv_INET_100M_V3, last_migrated_at=NOW, last_status_changed_at=NOW. sub-lm-commit-state flips PENDING_MIGRATION → ACTIVE.

June 1, 10:55:40 — EMITTING_EVENT: SubscriptionMigrated published. NOT-01 confirms to Otieno.

June 1, 10:55:42 — COMPLETED.

Wall time June 1: trigger 09:00 → committed 10:55:42 (~1h 56m). Subscription ACTIVE on source until target activation succeeded; PENDING_MIGRATION between source-suspend (09:00:57) and final-commit (10:55:40).

Alternative path — RESET_TO_OPERATION_DATE cycle anchor

If Otieno's request had cycleAnchorPolicy = RESET_TO_OPERATION_DATE, BIL-01 v1.1 returns 3 line items instead of 2: MIGRATION_FEE (1500), OLD_CYCLE_CREDIT (NEGATIVE — refund the source-price portion of cycle-days-already-paid; e.g., -1250 for half-cycle), FIRST_CYCLE_CHARGE (POSITIVE — full target Package price for new cycle starting at June 1; 7500). Totals net = 1500 - 1250 + 7500 + tax = ~8800. Otieno pays more upfront but the cycle anchor is now June 1.

Alternative path — same-tech migration without Package upgrade

A pure HomePass change with same Package would be a RELOCATION, not a MIGRATION. If a customer requests MIGRATION with targetPackageRef == sourcePackageRef, Drools rule "Target Package: same as source" rejects with SAME_PACKAGE. Caller redirected to RELOCATION-01.

Alternative path — cross-tech with CUSTOMER_REQUESTED_MIGRATION

If Otieno's target HomePass had a different network root type, Drools rule "Target HomePass: cross-tech with customer-requested trigger is rejected" fires -> 400 CROSS_Tech_CUSTOMER_REQUEST_NOT_ALLOWED. Backoffice may use ADMIN_Tech_MIGRATION only when FUL-10 is enabled for the operator and the customer belongs to an approved migration cohort.

Cancellation flow

Same as RELOCATION FLOW-WIK §4 — DELETE pending-migration emits external_cancel_requested, BPMN catches on timer boundary, emits SubscriptionMigrationCancelled, finalizes CANCELLED. Subscription stays on source.

WO timeouts and dispatcher cancellation

Same handling as RELOCATION FLOW-WIK §4 — WorkOrderTimedOut / WorkOrderCancelled triggers BPMN boundary, compensation chain (FUL-07 CLAIM back + FUL-04 RESUME at source, best-effort), Rejected event with appropriate rejectionReasonCode.

5. Cross-DD coordination

Module	Direction	Contract
SUB-WF-MIGRATION-01 (parent) v1.1	This satellite implements parent's contract	Triggers, validation, equipment-path + disconnect-mode + cycle-anchor + effective-timing
SUB-WF-FRAMEWORK-01 + WORKERS	This satellite composes framework primitives	12 workers composed
SUB-LM-01	Workers write status + master fields atomically	homepass_id, previous_homepass_id, package_ref, package_version_id, previous_package_ref, previous_package_version_id, last_migrated_at
RLM-CFG-01	Workers read source + target HomePass	service_management_endpoints, status_code, network_path root node type, franchise
SIP-01	Workers read source + target Packages	Composition, version, currency, billing mode
PLM-CFG-01 v1.0	Workers read service definitions	service_management_key, equipment requirements
BIL-01 v1.1	bil01-emit-intent	MIGRATION_INTENT with cycle-anchor context

Module	Direction	Contract
BIL-CFG-01 v1.0	BillableEvent catalog	MIGRATION_FEE, MIGRATION_PRORATE_DELTA (SIGNED), OLD_CYCLE_CREDIT, FIRST_CYCLE_CHARGE, EQUIPMENT_REPLACEMENT_FEE
FUL-04 v1.1	ful-call-suspend	SUSPEND_SERVICE at source with reasonCategory=SOURCE_DECOMMISSION
FUL-03 v1.1	ful-call-activate + ful-call-modify-service	ACTIVATE_SERVICE + MODIFY_SERVICE at target
FUL-07 v1.0	ful-call-release-homepass + ful-call-claim-homepass	HomePass bookkeeping
WO-01 (FRAMEWORK + FLOW-SHIFTING)	wo-create-shifting worker (when disconnectMode = TECH_DISPATCH)	Per DD_WO-01-CONTRACT-FOR-SUB-WF
FUL-10 Technology Migration Fulfillment	Parent operation delegates cross-tech ADMIN_TECH_MIGRATION	Owens technical cross-tech sequence and calls WO/OSR/PROV-INT as needed.
NOT-01	notification-send	Customer notification

5.1 Stub debt + open coordination items

- **WUG / WTZ / YASSN FLOW satellites** — pending operator deep dives.
- **FUL-10 operator enablement** — enables ADMIN_TECH_MIGRATION for approved cross-technology cohorts.
- **Operator-specific job-type code mapping** — operator's WO-01 Drools KJAR resolves (kind=SHIFTING, sourceNetworkRootType, equipmentPath, equipmentDelta) → operator job-type code.

B — Current TZ

Status: To be filled in after codebase cartography review.